

Week 5, Notebook 1: Attention & Transformers from Scratch

Pure Python + NumPy — The Architecture Behind GPT, BERT, and Modern AI

What you'll build: Self-attention, multi-head attention, and a complete Transformer encoder block — all in pure NumPy.

New concepts:

- Self-attention: every token attends to every other token
- Query, Key, Value decomposition
- Multi-head attention: parallel attention with different "perspectives"
- Positional encoding: giving the model a sense of order
- The Transformer block: Attention → Add & Norm → FFN → Add & Norm

Builds on: All previous weeks + GNN intuition (message passing $\approx$ attention)

Time estimate: 60–75 minutes

The Key Insight

A Transformer is a GNN where **every token is connected to every other token**, and the edge weights (attention) are **learned dynamically** from the data.

```
1 GNN: fixed graph structure, learned features
2 Transformer: learned graph structure (attention), learned features
```

Part 1: Self-Attention — The Core Mechanism

In [1]

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 np.random.seed(42)
4
5 # =====
6 # Self-attention from absolute zero
7 # =====
8 # Input: a sequence of token embeddings
9 # Output: a sequence of context-aware embeddings
10 #
11 # For each token:
12 #   1. Compute Query (what am I looking for?)
13 #   2. Compute Key (what do I contain?)
14 #   3. Compute Value (what information do I carry?)
15 #   4. Attention score = Query · KeyT / sqrt(dk)
16 #   5. Output = softmax(scores) · Values
17
18 def softmax(x, axis=-1):
19     exp_x = np.exp(x - x.max(axis=axis, keepdims=True))
20     return exp_x / exp_x.sum(axis=axis, keepdims=True)
21
22
23 class SelfAttention:
24     """Single-head self-attention."""
25
26     def __init__(self, d_model, d_k):
27         """
28         d_model: input embedding dimension
29         d_k: key/query dimension (also value dimension here)
30         """
31         scale = np.sqrt(2.0 / d_model)
32         self.W_Q = np.random.randn(d_model, d_k) * scale # Query projection
33         self.W_K = np.random.randn(d_model, d_k) * scale # Key projection
34         self.W_V = np.random.randn(d_model, d_k) * scale # Value projection
35         self.d_k = d_k
36
37     def forward(self, X):
38         """
39         X: (seq_len, d_model) – sequence of token embeddings
40         Returns: (seq_len, d_k) – context-aware embeddings
41         """
42         # Project to Q, K, V
43         Q = X @ self.W_Q # (seq_len, d_k) – what each token looks for
44         K = X @ self.W_K # (seq_len, d_k) – what each token advertises
45         V = X @ self.W_V # (seq_len, d_k) – what each token carries
46
47         # Attention scores: how much does token i attend to token j?
48         scores = (Q @ K.T) / np.sqrt(self.d_k) # (seq_len, seq_len)
49
50         # Softmax: normalize to get attention weights
51         self.attn_weights = softmax(scores) # (seq_len, seq_len)
52
53         # Weighted sum of values
54         output = self.attn_weights @ V # (seq_len, d_k)
55         return output
56
57
58 # Example: a 5-word sentence

```

```

59 tokens = ["The", "cat", "sat", "on", "mat"]
60 seq_len = len(tokens)
61 d_model = 8 # embedding dimension
62 d_k = 6 # attention dimension
63
64 # Random embeddings (in practice, these come from an embedding layer)
65 X = np.random.randn(seq_len, d_model)
66
67 # Run self-attention
68 attn = SelfAttention(d_model, d_k)
69 output = attn.forward(X)
70
71 print(f"Input shape: {X.shape} (seq_len x d_model)")
72 print(f"Output shape: {output.shape} (seq_len x d_k)")
73 print(f"Attention weights shape: {attn.attn_weights.shape} (seq_len x seq_len)")
74
75 # Visualize attention weights
76 fig, ax = plt.subplots(figsize=(6, 5))
77 im = ax.imshow(attn.attn_weights, cmap='Blues', vmin=0)
78 ax.set_xticks(range(seq_len))
79 ax.set_yticks(range(seq_len))
80 ax.set_xticklabels(tokens, fontsize=12)
81 ax.set_yticklabels(tokens, fontsize=12)
82 ax.set_xlabel('Attends TO (Keys)')
83 ax.set_ylabel('Attends FROM (Queries)')
84 ax.set_title('Self-Attention Weights')
85
86 for i in range(seq_len):
87     for j in range(seq_len):
88         ax.text(j, i, f'{attn.attn_weights[i,j]:.2f}',
89                 ha='center', va='center', fontsize=9,
90                 color='white' if attn.attn_weights[i,j] > 0.3 else 'black')
91
92 plt.colorbar(im, ax=ax, label='Attention Weight')
93 plt.tight_layout()
94 plt.savefig('w5_01_attention.png', dpi=100, bbox_inches='tight')
95 plt.show()
96
97 print("\nEach row shows: how much does that token attend to every other token.")
98 print("Row sums to 1.0 (softmax). This IS the 'attention' in 'Attention is All You Need'.")

```

Part 2: Multi-Head Attention

One attention head captures ONE type of relationship. **Multiple heads** capture different relationships in parallel.

"Head 1 might learn syntax, Head 2 might learn semantics, Head 3 might learn position."

Then we concatenate all heads and project back.

In [2]

```

1 # =====
2 # Multi-Head Attention
3 # =====
4
5 class MultiHeadAttention:
6     """Multi-head self-attention mechanism."""
7
8     def __init__(self, d_model, n_heads):
9         assert d_model % n_heads == 0, "d_model must be divisible by n_heads"
10        self.n_heads = n_heads
11        self.d_k = d_model // n_heads
12        self.d_model = d_model
13
14        # Each head has its own Q, K, V projections
15        self.heads = [SelfAttention(d_model, self.d_k) for _ in range(n_heads)]
16
17        # Output projection: concatenated heads back to d_model
18        scale = np.sqrt(2.0 / (self.d_k * n_heads))
19        self.W_O = np.random.randn(self.d_k * n_heads, d_model) * scale
20
21    def forward(self, X):
22        # Run all heads in parallel
23        head_outputs = [head.forward(X) for head in self.heads]
24
25        # Concatenate along feature dimension
26        concat = np.concatenate(head_outputs, axis=-1) # (seq_len, d_k * n_heads)
27
28        # Project back to d_model
29        output = concat @ self.W_O # (seq_len, d_model)
30        return output
31
32
33 # 4-head attention
34 mha = MultiHeadAttention(d_model=8, n_heads=4)
35 mha_output = mha.forward(X)
36
37 print(f"Multi-head attention output: {mha_output.shape}")
38 print(f"Number of heads: {mha.n_heads}, each with d_k = {mha.d_k}")
39
40 # Visualize all 4 heads
41 fig, axes = plt.subplots(1, 4, figsize=(20, 4))
42 for h in range(4):
43     weights = mha.heads[h].attn_weights
44     axes[h].imshow(weights, cmap='Blues', vmin=0, vmax=weights.max())
45     axes[h].set_title(f'Head {h+1}')
46     axes[h].set_xticks(range(seq_len))
47     axes[h].set_yticks(range(seq_len))
48     axes[h].set_xticklabels(tokens, fontsize=9, rotation=45)
49     axes[h].set_yticklabels(tokens, fontsize=9)
50
51 plt.suptitle('MULTI-HEAD ATTENTION: 4 Different Attention Patterns', fontsize=14, fontweight='bold')
52 plt.tight_layout()
53 plt.savefig('w5_01_multihead.png', dpi=100, bbox_inches='tight')
54 plt.show()
55
56 print("\nEach head learns a DIFFERENT attention pattern!")
57 print("This is like having 4 different 'perspectives' on the same data.")

```

Part 3: The Complete Transformer Block

A full Transformer encoder block:

```
1 Input → Multi-Head Attention → Add & Norm → FFN → Add & Norm → Output
2   ↑_____|                               ↑_____|
3   (residual connection)                   (residual connection)
```

In [3]

```

1 # =====
2 # Layer Normalization
3 # =====
4
5 class LayerNorm:
6     """Layer normalization (Ba et al., 2016)."""
7
8     def __init__(self, d_model, eps=1e-6):
9         self.gamma = np.ones(d_model)
10        self.beta = np.zeros(d_model)
11        self.eps = eps
12
13    def forward(self, x):
14        mean = x.mean(axis=-1, keepdims=True)
15        std = x.std(axis=-1, keepdims=True)
16        return self.gamma * (x - mean) / (std + self.eps) + self.beta
17
18
19 # =====
20 # Feed-Forward Network (position-wise)
21 # =====
22
23 class FeedForward:
24     """Position-wise feed-forward network."""
25
26     def __init__(self, d_model, d_ff):
27         scale1 = np.sqrt(2.0 / d_model)
28         scale2 = np.sqrt(2.0 / d_ff)
29         self.W1 = np.random.randn(d_model, d_ff) * scale1
30         self.b1 = np.zeros(d_ff)
31         self.W2 = np.random.randn(d_ff, d_model) * scale2
32         self.b2 = np.zeros(d_model)
33
34    def forward(self, x):
35        h = np.maximum(0, x @ self.W1 + self.b1) # ReLU
36        return h @ self.W2 + self.b2
37
38
39 # =====
40 # Positional Encoding
41 # =====
42
43 def positional_encoding(seq_len, d_model):
44     """Sinusoidal positional encoding (Vaswani et al., 2017)."""
45     PE = np.zeros((seq_len, d_model))
46     for pos in range(seq_len):
47         for i in range(0, d_model, 2):
48             PE[pos, i] = np.sin(pos / 10000 ** (i / d_model))
49             if i + 1 < d_model:
50                 PE[pos, i+1] = np.cos(pos / 10000 ** (i / d_model))
51     return PE
52
53
54 # =====
55 # Complete Transformer Encoder Block
56 # =====
57
58 class TransformerBlock:

```

```

59     """One Transformer encoder block."""
60
61     def __init__(self, d_model, n_heads, d_ff):
62         self.mha = MultiHeadAttention(d_model, n_heads)
63         self.norm1 = LayerNorm(d_model)
64         self.ff = FeedForward(d_model, d_ff)
65         self.norm2 = LayerNorm(d_model)
66
67     def forward(self, X):
68         # Multi-head attention + residual + norm
69         attn_out = self.mha.forward(X)
70         X = self.norm1.forward(X + attn_out) # Residual connection
71
72         # Feed-forward + residual + norm
73         ff_out = self.ff.forward(X)
74         X = self.norm2.forward(X + ff_out) # Residual connection
75
76         return X
77
78
79 # Build a 2-block Transformer encoder
80 d_model = 16
81 n_heads = 4
82 d_ff = 64
83 seq_len = 8
84
85 # Random input embeddings + positional encoding
86 X_seq = np.random.randn(seq_len, d_model) * 0.5
87 PE = positional_encoding(seq_len, d_model)
88 X_input = X_seq + PE
89
90 # Stack 2 Transformer blocks
91 block1 = TransformerBlock(d_model, n_heads, d_ff)
92 block2 = TransformerBlock(d_model, n_heads, d_ff)
93
94 h = block1.forward(X_input)
95 h = block2.forward(h)
96
97 print(f"Input: {X_input.shape} (seq_len x d_model)")
98 print(f"Output: {h.shape} (seq_len x d_model)")
99 print(f"\nTransformer block components:")
100 print(f"  Multi-Head Attention: {n_heads} heads, d_k = {d_model // n_heads}")
101 print(f"  Feed-Forward: {d_model} -> {d_ff} -> {d_model}")
102 print(f"  Positional Encoding: sinusoidal")
103 print(f"\nTotal blocks: 2 (each token has seen the full sequence TWICE)")

```

In [4]

```

1 # Visualize positional encoding
2 PE_vis = positional_encoding(50, 32)
3
4 fig, axes = plt.subplots(1, 2, figsize=(14, 5))
5
6 axes[0].imshow(PE_vis.T, cmap='RdBu_r', aspect='auto')
7 axes[0].set_xlabel('Position')
8 axes[0].set_ylabel('Dimension')
9 axes[0].set_title('Positional Encoding Matrix')
10
11 for dim in [0, 1, 2, 3, 6, 10]:
12     axes[1].plot(PE_vis[:, dim], label=f'dim {dim}', alpha=0.8)
13 axes[1].set_xlabel('Position')
14 axes[1].set_ylabel('Value')
15 axes[1].set_title('PE by Dimension (sinusoidal waves)')
16 axes[1].legend(fontsize=8)
17 axes[1].grid(True, alpha=0.2)
18
19 plt.suptitle('POSITIONAL ENCODING: Giving Order to the Transformer', fontsize=14, fontweight='bold')
20 plt.tight_layout()
21 plt.savefig('w5_01_positional.png', dpi=100, bbox_inches='tight')
22 plt.show()
23
24 print("Lower dimensions = high frequency (position changes fast)")
25 print("Higher dimensions = low frequency (captures longer-range position)")
26 print("This lets the model learn both local and global position relationships.")

```

■ Self-Check

- [] You can explain attention in one sentence: "Each token computes a weighted average of all tokens' values, weighted by query-key similarity"
- [] You understand Q, K, V: Query = "what I want", Key = "what I offer", Value = "what I carry"
- [] You can explain multi-head: "parallel attention heads capture different relationship types"
- [] You know why positional encoding exists: attention is permutation-invariant, PE breaks symmetry
- [] You see the connection: GNN message passing is attention with a fixed graph; Transformers learn the graph

➡■ **Next:** `w5_02_PyTorch_Transformer.ipynb` — **Build a real Transformer in PyTorch**